React.js

From Zero to Advanced

A complete, example-driven guide — from your first component to production-grade patterns.

JSX · Components · Hooks · State Management · Performance · Routing · Advanced Patterns

Contents

1	Why React? & Setting Up
2	JSX — JavaScript + XML
3	Components & Props
4	State with useState
5	Events & Forms
6	Conditional Rendering & Lists
7	useEffect — Side Effects & Data Fetching
8	Context API — Avoiding Prop Drilling
9	useReducer & State Machines
10	Custom Hooks
11	useRef, useMemo & useCallback
12	Performance — memo, lazy, Suspense
13	Routing with React Router
14	Advanced Patterns & Error Boundaries

1 · Why React? & Setting Up

React is a JavaScript library for building user interfaces using a **component-based**, **declarative** model. You describe *what* the UI should look like for a given state, and React figures out *how* to update the DOM efficiently.

Core ideas

- **Components** — small, reusable building blocks that return UI.
- **Declarative UI** — UI = f(state). Change state, React re-renders.
- **Virtual DOM** — React diffs changes and touches the real DOM minimally.
- **Unidirectional data flow** — data flows down (props), events flow up.

Creating a project

The standard toolchain is **Vite** (fast) or Create React App (older). With Node.js installed:

```
# Create a new project (Vite — recommended)
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev        # starts dev server at http://localhost:5173
```

A React app mounts at a single DOM node:

```
// main.jsx — the entry point
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)
```

■ StrictMode runs checks and double-invokes render functions in development to surface bugs early. Keep it on.

2 · JSX — JavaScript + XML

JSX is a syntax extension that lets you write HTML-like markup inside JavaScript. It is not a string — it compiles to function calls (`React.createElement`).

Rules of JSX

- Return **one parent element** — wrap siblings in a tag or a fragment `<>...</>`.
- Use **className** instead of class; **htmlFor** instead of for.
- Attributes are camelCase: `onClick`, `tabIndex`, `strokeWidth`.
- JavaScript expressions go inside curly braces `{ }`.
- Every tag must close: `<img />`, `<br />`.

Example

```
function Greeting({ name }) {
  const hour = new Date().getHours()
  const timeOfDay = hour < 12 ? 'morning' : hour < 18 ? 'afternoon' : 'evening'

  return (
    <div className="card">
      <h1>Good {timeOfDay}, {name}!</h1>
      <p>Today is {new Date().toLocaleDateString()}</p>
      <img src="/logo.png" alt="Logo" />
    </div>
  )
}
```

■ **JSX prevents injection attacks. Anything you interpolate is escaped — never use dangerouslySetInnerHTML unless you fully trust the content.**

Styling

```
// 1. CSS file
import './Greeting.css'

// 2. Inline style object (camelCase properties)
const style = { color: 'tomato', fontSize: 20, marginTop: 8 }
<p style={style}>Inline styled</p>

// 3. CSS Modules — scoped class names
import styles from './Card.module.css'
<div className={styles.card}>Scoped</div>
```

3 · Components & Props

A component is a JavaScript function (or class) that returns JSX. **Props** are read-only inputs passed from a parent — like function parameters. A component must never modify its own props.

Function components (the standard)

```
// Button.jsx
export default function Button({ label, color = 'blue', onClick, children }) {
  return (
    <button style={{ background: color }} onClick={onClick}>
      {children ?? label}
    </button>
  )
}
```

Composition: children & props

```
// App.jsx
import Button from './Button'

export default function App() {
  return (
    <div>
      <Button label="Save" color="green" onClick={() => alert('Saved!')} />
      <Button onClick={() => alert('Deleted!')}>
        Delete             {/* children slot */}
      </Button>
    </div>
  )
}
```

Props vs State

■ **Think of props as function arguments and state as local variables that can trigger a re-render when changed.**

4 · State with useState

State is data a component remembers between renders. The `useState` hook returns the current value and a setter function. Calling the setter schedules a re-render.

```
import { useState } from 'react'

function Counter() {
  const [count, setCount] = useState(0)   // initial value: 0

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  )
}
```

Functional updates (important!)

When the new state depends on the previous state, pass a function. React batches updates, so `setCount(count + 1)` twice would only add 1.

```
setCount(prev => prev + 1)
setCount(prev => prev + 1)   // now adds 2 — always correct
```

Objects & arrays — immutability

■ **Never mutate state directly (count++, obj.x = 1, arr.push). Always create a new object/array so React detects the change.**

```
const [user, setUser] = useState({ name: 'Ada', age: 28 })

// ■ Wrong
user.age = 29

// ■ Right
setUser(prev => ({ ...prev, age: 29 }))

const [todos, setTodos] = useState([])

// ■ Add
setTodos(prev => [...prev, { id: 1, text: 'Learn React', done: false }])

// ■ Update one item
setTodos(prev => prev.map(t => t.id === 1 ? { ...t, done: !t.done } : t))

// ■ Remove
setTodos(prev => prev.filter(t => t.id !== 1))
```

■ **Rule of Hooks: only call hooks at the top level of a component (not inside loops, conditions, or nested functions), and only from React functions.**

5 · Events & Forms

Handling events

React events are camelCase synthetic events. Pass a function reference — do not call it in JSX.

```
function Search() {
  const handleSubmit = (e) => {
    e.preventDefault()               // stop page reload
    console.log('Submitted')
  }

  return (
    <form onSubmit={handleSubmit}>
      <input onChange={(e) => console.log(e.target.value)} />
      <button type="submit">Go</button>
    </form>
  )
}
```

Controlled components

A **controlled input** gets its value from state and updates it via onChange. React is the single source of truth — this is the recommended pattern.

```
function LoginForm() {
  const [form, setForm] = useState({ email: '', password: '' })

  const handleChange = (e) => {
    const { name, value } = e.target
    setForm(prev => ({ ...prev, [name]: value }))   // dynamic key
  }

  const handleSubmit = (e) => {
    e.preventDefault()
    console.log(form)
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="email" type="email" value={form.email}
             onChange={handleChange} placeholder="Email" />
      <input name="password" type="password" value={form.password}
             onChange={handleChange} placeholder="Password" />
      <button disabled={!form.email || !form.password}>Log in</button>
    </form>
  )
}
```

■ Reuse one handleChange for all fields with a computed property name [name]: value — no per-field handlers needed.

6 · Conditional Rendering & Lists

Conditional rendering

```
function Dashboard({ user, notifications }) {
  return (
    <div>
      {/* if / else */}
      {user ? <h1>Welcome, {user.name}</h1> : <h1>Please log in</h1>}

      {/* if only (&&) — renders nothing when false */}
      {notifications.length > 0 && (
        <span className="badge">{notifications.length} new</span>
      )}

      {/* ternary inside JSX for either/or UI */}
      {user.isAdmin
        ? <AdminPanel />
        : <UserPanel />}
    </div>
  )
}
```

■ **A && B returns 0 when A is 0 — {count && "items"} renders "0". Use {count > 0 && "items"} instead.**

Rendering lists with keys

```
function TodoList({ todos, onToggle }) {
  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>              {/* key = stable, unique id */}
          <label>
            <input type="checkbox" checked={todo.done}
                   onChange={() => onToggle(todo.id)} />
            <span style={{ textDecoration: todo.done ? 'line-through' : 'none' }}>
              {todo.text}
            </span>
          </label>
        </li>
      ))}
    </ul>
  )
}
```

Keys let React match items between renders. Use stable unique ids — never array indexes when the list can reorder, be filtered, or items can be inserted/removed, or you will get subtle UI bugs.

Lifting state up

When two siblings need the same data, move the state to their closest common parent and pass it down as props.

```
function App() {
  const [query, setQuery] = useState('')

  return (
    <>
      <SearchBar query={query} onQueryChange={setQuery} />
      <Results query={query} />
```

```
      </>
    )
  }
```

7 · useEffect — Side Effects & Data Fetching

Side effects are anything outside pure rendering: network requests, timers, subscriptions, DOM mutations, localStorage. `useEffect` runs after the render commits, keeping rendering pure.

```
import { useState, useEffect } from 'react'

function UserProfile({ userId }) {
  const [user, setUser] = useState(null)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)

  useEffect(() => {
    const controller = new AbortController()      // for cancellation

    async function load() {
      try {
        setLoading(true)
        const res = await fetch(`/api/users/${userId}`,
                                { signal: controller.signal })
        if (!res.ok) throw new Error(res.statusText)
        setUser(await res.json())
      } catch (err) {
        if (err.name !== 'AbortError') setError(err.message)
      } finally {
        setLoading(false)
      }
    }

    load()
    return () => controller.abort()               // cleanup on unmount/re-run
  }, [userId])                                    // re-runs when userId changes

  if (loading) return <p>Loading…</p>
  if (error)   return <p>Error: {error}</p>
  return <h1>{user.name}</h1>
}
```

The dependency array

```
useEffect(() => { ... })        // runs after EVERY render
useEffect(() => { ... }, [])    // runs ONCE on mount
useEffect(() => { ... }, [a, b])// runs when a or b change
```

Cleanup function

```
useEffect(() => {
  const id = setInterval(() => tick(), 1000)
  return () => clearInterval(id)   // clear timers, subscriptions, listeners
}, [])
```

■ **Do not fetch in render. Fetch in useEffect (or event handlers / data-fetching libraries like TanStack Query). Skipping the dependency array when you use values inside the effect is the #1 source of stale-closure bugs — React's ESLint plugin will warn you.**

8 · Context API — Avoiding Prop Drilling

Context passes data (theme, auth user, language, cart) deep into the tree without threading props through every level.

1. Create the context

```
import { createContext, useContext, useState } from 'react'

const ThemeContext = createContext('light')
```

2. Provide a value

```
function App() {
  const [theme, setTheme] = useState('light')

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Header />
      <Page />
    </ThemeContext.Provider>
  )
}
```

3. Consume it anywhere below

```
import { useContext } from 'react'

function Header() {
  const { theme, setTheme } = useContext(ThemeContext)

  return (
    <header style={{ background: theme === 'dark' ? '#111' : '#fff' }}>
      <button onClick={() => setTheme(t => t === 'dark' ? 'light' : 'dark')}>
        Toggle theme
      </button>
    </header>
  )
}
```

■ **Keep contexts focused and small. Splitting "user" and "theme" into two providers prevents unrelated consumers from re-rendering.**

Custom Provider + hook (production pattern)

```
// ThemeContext.js
import { createContext, useContext, useState } from 'react'

const ThemeContext = createContext(null)

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light')
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      {children}
    </ThemeContext.Provider>
  )
}

export function useTheme() {
  const ctx = useContext(ThemeContext)
```

```
  if (!ctx) throw new Error('useTheme must be used inside <ThemeProvider>')
  return ctx
}
```

9 · useReducer & State Machines

`useReducer` manages complex state transitions with a **reducer** function: (state, action) => newState. It centralizes logic, makes updates predictable, and pairs naturally with Context.

Shopping cart example

```
import { useReducer } from 'react'

const initialState = { items: [], total: 0 }

function reducer(state, action) {
  switch (action.type) {
    case 'add': {
      const item = state.items.find(i => i.id === action.payload.id)
      const items = item
        ? state.items.map(i => i.id === item.id
            ? { ...i, qty: i.qty + 1 } : i)
        : [...state.items, { ...action.payload, qty: 1 }]
      return { items, total: computeTotal(items) }
    }
    case 'remove':
      return { ...state, items: state.items.filter(i => i.id !== action.id) }
    case 'clear':
      return initialState
    default:
      throw new Error(`Unknown action: ${action.type}`)
  }
}

function Cart() {
  const [state, dispatch] = useReducer(reducer, initialState)

  return (
    <div>
      <p>Total: ${state.total.toFixed(2)}</p>
      <button onClick={() =>
        dispatch({ type: 'add', payload: { id: 1, name: 'Book', price: 12 } })}>
        Add book
      </button>
      <button onClick={() => dispatch({ type: 'clear' })}>Clear</button>
    </div>
  )
}
```

When to choose which

■ **Reducers must be pure: same (state, action) always gives the same new state, with no side effects.**

10 · Custom Hooks

A custom hook is a function that starts with `use` and may call other hooks. It extracts and **reuses stateful logic** — not UI. The same hook can be used by a component, a form, a test…

useLocalStorage — persist state across reloads

```
import { useState, useEffect } from 'react'

function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    try {
      const stored = localStorage.getItem(key)
      return stored !== null ? JSON.parse(stored) : initialValue
    } catch {
      return initialValue
    }
  })

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value))
  }, [key, value])

  return [value, setValue]
}

// Usage — feels exactly like useState, but survives refreshes
function Settings() {
  const [name, setName] = useLocalStorage('username', 'Guest')
  return <input value={name} onChange={e => setName(e.target.value)} />
}
```

useDebounce — wait for typing to stop

```
import { useState, useEffect } from 'react'

function useDebounce(value, delay = 400) {
  const [debounced, setDebounced] = useState(value)

  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay)
    return () => clearTimeout(id)
  }, [value, delay])

  return debounced
}

// Usage — fetch search results only after the user pauses
function SearchPage() {
  const [query, setQuery] = useState('')
  const debouncedQuery = useDebounce(query)

  useEffect(() => {
    if (debouncedQuery) fetchResults(debouncedQuery)
  }, [debouncedQuery])

  return <input value={query} onChange={e => setQuery(e.target.value)} />
}
```

■ **If you find yourself copying the same useState + useEffect pair into multiple components, that logic belongs in a custom hook.**

11 · useRef, useMemo & useCallback

useRef — a mutable box that survives renders

A ref holds a value that persists across renders without causing re-renders. Classic uses: DOM access and previous values.

```
import { useRef, useEffect } from 'react'

function AutoFocusInput() {
  const inputRef = useRef(null)

  useEffect(() => {
    inputRef.current.focus()          // access the DOM node
  }, [])

  const renderCount = useRef(0)
  renderCount.current += 1            // count renders without re-rendering

  return <input ref={inputRef} />
}
```

useMemo — cache expensive computations

```
import { useMemo } from 'react'

function Report({ transactions, filter }) {
  const total = useMemo(() => {
    return transactions
      .filter(t => t.category === filter)
      .reduce((sum, t) => sum + t.amount, 0)   // expensive
  }, [transactions, filter])                   // recompute only when these change

  return <h2>Total: ${total}</h2>
}
```

useCallback — cache function identities

Functions are recreated every render, breaking memoized children. `useCallback` keeps the same function reference.

```
import { useCallback } from 'react'

function ProductList({ products, onAdd }) {
  const handleAdd = useCallback((id) => {
    onAdd(id)                                  // stable reference
  }, [onAdd])

  return products.map(p =>
    <ProductRow key={p.id} product={p} onAdd={handleAdd} />)
}
```

■ **Memoization is not free. Reach for useMemo/useCallback when profiling shows real cost, or when a stable reference is required (dependency arrays of other hooks, React.memo children). Premature memoization adds complexity for no gain.**

12 · Performance — memo, lazy, Suspense

React.memo — skip unchanged renders

```
import { memo } from 'react'

// Re-renders only when its own props change
const ProductRow = memo(function ProductRow({ product, onAdd }) {
  return (
    <tr>
      <td>{product.name}</td>
      <td><button onClick={() => onAdd(product.id)}>Add</button></td>
    </tr>
  )
})
```

When a parent re-renders, React re-renders all children by default. `memo` compares props (shallow) and bails out if nothing changed — critical for long lists.

Code splitting with lazy + Suspense

```
import { lazy, Suspense } from 'react'

// Loaded on demand — the initial bundle shrinks
const AdminPanel = lazy(() => import('./AdminPanel'))

function App() {
  return (
    <Suspense fallback={<Spinner />}>
      <AdminPanel />
    </Suspense>
  )
}
```

useTransition — keep UI responsive

```
import { useTransition, useState } from 'react'

function FilterList({ items }) {
  const [query, setQuery] = useState('')
  const [filtered, setFiltered] = useState(items)
  const [isPending, startTransition] = useTransition()

  const handleChange = (e) => {
    setQuery(e.target.value)                  // urgent: input stays snappy
    startTransition(() => {                   // non-urgent: list may lag
      setFiltered(items.filter(i => i.includes(e.target.value)))
    })
  }

  return (
    <>
      <input value={query} onChange={handleChange} />
      {isPending && <small>Filtering…</small>}
      <List items={filtered} />
    </>
  )
}
```

- **Memoize children** of frequently re-rendering parents.

- **Split routes** with lazy + Suspense so users download only the code they visit.
- **Virtualize** lists of 1000+ rows (react-window).
- **Measure first** with the React DevTools Profiler tab — optimize what is slow.

13 · Routing with React Router

A single-page app needs client-side routing. **React Router** maps URLs to components without page reloads.

```
import { BrowserRouter, Routes, Route, Link, NavLink,
         useParams, useNavigate } from 'react-router-dom'

function App() {
  return (
    <BrowserRouter>
      <nav>
        <NavLink to="/">Home</NavLink>          {/* active styling built-in */}
        <NavLink to="/products">Products</NavLink>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/products" element={<ProductList />} />
        <Route path="/products/:id" element={<ProductDetail />} />
        <Route path="*" element={<NotFound />} /> {/* 404 catch-all */}
      </Routes>
    </BrowserRouter>
  )
}
```

URL parameters & navigation

```
import { useParams, useNavigate, useSearchParams } from 'react-router-dom'

function ProductDetail() {
  const { id } = useParams()                    // /products/42 → id = "42"
  const navigate = useNavigate()
  const [searchParams, setSearchParams] = useSearchParams()

  return (
    <div>
      <h1>Product {id}</h1>
      <p>Sort: {searchParams.get('sort')}</p>
      <button onClick={() => setSearchParams({ sort: 'price' })}>Sort by price</button>
      <button onClick={() => navigate(-1)}>← Back</button>
    </div>
  )
}
```

Protected routes

```
import { Navigate, Outlet } from 'react-router-dom'

function RequireAuth() {
  const { user } = useAuth()
  return user ? <Outlet /> : <Navigate to="/login" replace />
}

// In Routes:
// <Route element={<RequireAuth />}>
//   <Route path="/dashboard" element={<Dashboard />} />
// </Route>
```

■ **Install with: npm install react-router-dom. Use BrowserRouter for pretty URLs (needs server config on static hosts) or HashRouter to avoid server setup.**

14 · Advanced Patterns & Error Boundaries

Render props

A component receives a function as a child/prop and calls it with data — inverting control to the consumer.

```
function MouseTracker({ render }) {
  const [position, setPosition] = useState({ x: 0, y: 0 })

  return (
    <div onMouseMove={e =>
      setPosition({ x: e.clientX, y: e.clientY })}>
      {render(position)}
    </div>
  )
}

// Usage
<MouseTracker render={({ x, y }) => <p>Mouse: {x}, {y}</p>} />
```

Compound components (flexible APIs)

```
// Tabs.jsx — parent coordinates via context, children stay declarative
import { createContext, useContext, useState } from 'react'

const TabsContext = createContext(null)

function Tabs({ children, defaultTab }) {
  const [active, setActive] = useState(defaultTab)
  return (
    <TabsContext.Provider value={{ active, setActive }}>
      <div className="tabs">{children}</div>
    </TabsContext.Provider>
  )
}

function Tab({ id, children }) {
  const { active, setActive } = useContext(TabsContext)
  return (
    <button className={active === id ? 'active' : ''}
            onClick={() => setActive(id)}>
      {children}
    </button>
  )
}

function Panel({ id, children }) {
  const { active } = useContext(TabsContext)
  return active === id ? <div>{children}</div> : null
}

// Beautiful, self-describing usage:
<Tabs defaultTab="profile">
  <Tab id="profile">Profile</Tab>
  <Tab id="settings">Settings</Tab>
  <Panel id="profile"><UserForm /></Panel>
  <Panel id="settings"><SettingsForm /></Panel>
</Tabs>
```

Error boundaries

Error boundaries are class components that catch JavaScript errors in their child tree and show a fallback UI instead of crashing the whole app.

```
import { Component } from 'react'

class ErrorBoundary extends Component {
  state = { hasError: false, error: null }

  static getDerivedStateFromError(error) {
    return { hasError: true, error }
  }

  componentDidCatch(error, info) {
    console.error('Caught:', error, info.componentStack)
    // send to an error-reporting service (Sentry, etc.)
  }

  render() {
    if (this.state.hasError) {
      return <h2>Something went wrong. Please refresh.</h2>
    }
    return this.props.children
  }
}

// Usage — wrap fragile subtrees
<ErrorBoundary>
  <PaymentWidget />
</ErrorBoundary>
```

■ **Wrap route-level sections and risky widgets in error boundaries. They catch rendering errors only — not event handlers or async code (use try/catch there).**

Where to go next

- **Data fetching** — TanStack Query or SWR: caching, deduping, retries out of the box.
- **Global state** — Zustand or Redux Toolkit when Context is not enough.
- **Forms** — React Hook Form for performant, validation-ready forms.
- **Server components** — Next.js / React Server Components for rendering on the server.
- **Testing** — Vitest + React Testing Library (test behavior, not implementation).
- **Types** — TypeScript with React gives autocomplete and compile-time safety.

Keep components small and pure. Lift state only when needed. And when in doubt — the React docs (react.dev) are excellent.